RepoMind — Consolidated Reference for the Research Paper

Compiled from `docs/PROJECT.md`, `docs/DATA_MODEL.md`, `docs/METRICS.md`, `docs/RISK.md`, `docs/EVALUATION.md` + `EVALUATION_RESULTS.md`, `docs/RETRIEVAL.md`, `docs/LLM.md`, `docs/EXPLAIN.md`, `docs/ASSIST.md`, `docs/UI.md`, and the project's own `CLAUDE.md`. Nothing below is invented — every number and claim traces back to one of those files or to the code in the module it describes. Where a result is a hypothesis rather than a finding, that is stated the same way the source docs state it.

---

1. Title / framing

RepoMind: A Repository-Intelligence System for Evidence-Grounded, Knowledge-Aware Software History Analysis

One-line description: RepoMind mines Git and GitHub history to build a structured knowledge model of a repository — who worked on what, where contribution is concentrated, where maintenance risk may exist — and uses that structured evidence to ground LLM answers to repository-specific questions, rather than asking an LLM to reason from source code alone.

Core architectural principle, stated once in the original build plan and carried through every module:

```
Data -> Structured Knowledge -> Retrieval -> LLM -> Repository Intelligence
```

The LLM is the last stage, not the foundation.

2. Problem statement and research questions

Generic code assistants explain *what* code does but not *why* it looks the way it does, who touched it, or what historical risk surrounds it. RepoMind treats a repository's commit history, pull requests, issues, and reviews as primary research data, not as context to glue onto a prompt.

Five research questions were defined up front and are revisited explicitly in the Module 6 evaluation (Section 8 below):

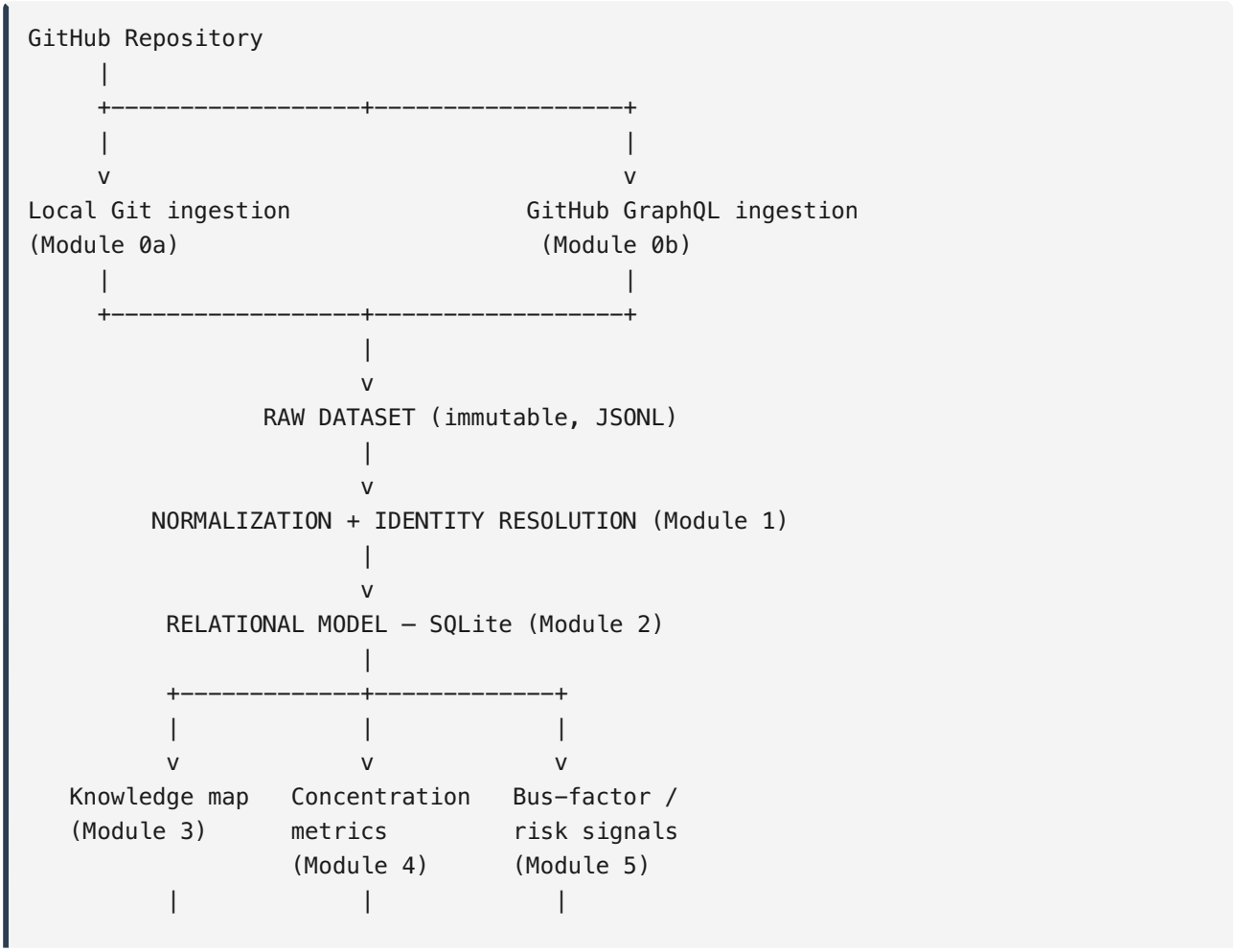
#	Research question
RQ1	How can heterogeneous Git and GitHub evidence be transformed into a structured repository knowledge model?
RQ2	Which activity signals are useful for estimating knowledge concentration within software components?
RQ3	How does identity resolution affect developer-level knowledge analysis?
RQ4	Can repository-aware retrieval improve the factual grounding of LLM answers about software history?
RQ5	Can knowledge-concentration signals identify potential maintenance-risk areas?

Status of each, in one line (full discussion in Section 8):

- **RQ1 — Supported.** The pipeline runs end-to-end on 8 heterogeneous repositories with no structural failures; the specific linkage gaps are quantified rather than papered over.
- **RQ2 — Partly answered.** Commit-derived signals are available everywhere; PR/review-derived signals are only usable where the merge strategy preserves commit linkage (a strong caveat, detailed below).
- **RQ3 — Partly answered.** The *within-source* resolution (same-name merging) barely changes rankings; the real problem — linking a Git identity to a GitHub identity — remains largely unsolved and its quantitative effect on the metrics is explicitly flagged as unmeasured.
- **RQ4 — Not evaluated at scale.** The grounding mechanism is built and works on a small hand-checked and a 10-question generated sample; no large evaluation of "does retrieval improve grounding" (e.g. vs. no retrieval) has been run.
- **RQ5 — Not answered.** Nothing in the current evaluation checks whether components flagged as high-risk actually became maintenance problems later. That would require a temporal backtest, explicitly listed as future work.

This honesty about what is and isn't shown is itself a methodological point worth making in the paper: the project treats every derived score as a hypothesis to be tested, never as ground truth about people.

3. System architecture



```
          +-------------+-------------+
                        |
                        v
          MULTI-REPO EVALUATION (Module 6)
                        |
                        v
      STRUCTURED + TEXT RETRIEVAL (Module 7)
                        |
                        v
          GROUNDED LLM (Gemini) (Module 8)
                 /             \
                v               v
    CODE EXPLANATION (9)   ENGINEERING ASSISTANCE (10)
                        |
                        v
                  WEB UI (Module 11)
```

4. Module-by-module description

Module 0a — Local Git ingestion (`ingest/`)

Clones a public repository's full history and walks `HEAD` with GitPython, writing one JSON record per commit to `data/raw/<repo>/commits.jsonl`: SHA, parent SHAs, author/committer (name + email), authored/committed timestamps, subject/body, and per-file insertion/deletion counts.

Documented, inherited limitations (not pipeline bugs): extraction follows the default branch/HEAD only; merge commits frequently carry an empty file list; renames appear in git's own raw form rather than an explicit `old_path -> new_path` mapping.

Module 0b — GitHub GraphQL acquisition (`ingest/github_*`)

Paginated GraphQL acquisition of issues, pull requests (with nested reviews), and labels, with resumable checkpointing (`checkpoint.json` records the last cursor per resource). Authentication uses `GITHUB_TOKEN` from a gitignored `.env`, loaded via `python-dotenv`; the token is never logged, printed, or written into any raw dataset.

Module 1 — Normalization & identity resolution (`normalization/`)

Converts raw JSONL into normalized entities (Repository, Developer, File, Commit, CommitFile, Issue, PullRequest, Review, Label — exact fields in `normalization/schema.py`) with a validator that checks for dangling references before anything is loaded further downstream.

Identity resolution (the project's most methodologically interesting piece, and the direct empirical basis for RQ3) is deliberately conservative — an identity is merged only on a concrete signal:

1. **GitHub no-reply commit email** (e.g. `49699333+dependabot[bot]@users.noreply.github.com`) directly encodes a GitHub login — the *only* signal strong enough to link a Git identity to a GitHub identity here, since GitHub itself generated the address. Confidence: **high**.
2. **Same email (case-insensitive)** clusters commits to one developer. Confidence: **medium**.

3. **No email present** falls back to clustering by raw display-name string. Confidence: **low** (two people can share a name).
4. **GitHub-only identities** (a PR/issue/review author with no matching commit email) get their own developer keyed by login. Confidence: **medium**, source `github_login_only`.

Every `Developer` record carries `identity_confidence` and `identity_sources` so later analysis can be honest about resolution quality instead of presenting resolved identities as fact. Bots (`[bot]` suffix, `dependabot`, `github-actions`, etc.) are flagged via `is_bot` rather than deleted, so they can be excluded from human-knowledge metrics without losing the record.

Module 2 — Database & relationship construction (`database/`)

Normalized entities are loaded into SQLite (`database/schema.sql`), a derived, fully regeneratable artifact. Two materialized edge tables (`pr_commit`, `pr_issue`) make the many-to-many PR↔Commit and PR↔Issue relationships directly joinable even though they're stored as JSON arrays on `pull_request` for provenance. Every normalized record carries a `raw_source_reference` back to the exact raw JSONL record it came from (a commit SHA, or `<file>.jsonl#<github_number>`) — full provenance from raw evidence to query result.

Module 3 — Knowledge map visualization (`visualization/`)

An interactive developer↔file graph (HTML, `data/<repo>_knowledge_map.html`) built directly from the relational model — the first concrete check that Developer→Commit→File (and its PR/Review/Issue extensions) is actually queryable and sensible, before any metric is built on top of it.

Module 4 — Knowledge concentration metrics (`analysis/`)

Explicit framing: nothing here measures "expertise"; every number is a signal derived from activity, and the combined score is a hypothesis to be evaluated, not a fact about people. Components are derived from path prefixes (depth configurable; default 2), the plan's documented stand-in for a first-class Module entity.

Seven **signals** per developer per component: commits (distinct commits authored), lines (additions+deletions), recency (exponential decay, default half-life 365 days, reference date = the repo's last commit so results are reproducible), continuity (distinct calendar years active), PRs authored touching the component, reviews given (excluding self-review), and issues closed by the developer's PRs touching the component.

Five candidate concentration metrics, all turning a per-developer quantity into a share `s_i` first:

- **A — Contribution share**: raw share from commit counts.
- **B — Concentration index**: Herfindahl–Hirschman `HHI = Σ s_i²` (1 = fully concentrated, 1/n = perfectly even).
- **C — Effective contributor count**: `1 / HHI` — the number of equally-active contributors that would produce the same concentration. This is the human-readable number (e.g. "7.0 effective contributors" vs. "HHI = 0.14").
- **D — Recency-weighted concentration**: A/B/C recomputed from recency weights instead of raw commit counts.

- **E — Multi-signal score**: weighted mean of a developer's share across *all* signals that have data in that component (a missing signal is dropped, not treated as zero, so missing GitHub data doesn't silently skew it); weights default to equal and are recorded in the output JSON. Explicitly labeled a **research hypothesis**, not a validated score.

Components under a minimum-commit threshold (default 5) are reported but excluded from ranking, since one commit trivially produces HHI = 1.

Key limitation surfaced by the project itself: because of the identity-resolution conservatism above, a developer whose commits aren't linked to their GitHub login has their commit activity and their PR/review activity split across two separate `Developer` records. In flask this is concretely "David Lord" (commits) vs. `davidism` (PRs/reviews) — the same person, measured as two. This understates concentration, especially for metric E, and is called out as the direct evidence behind RQ3 and the strongest argument for improving identity resolution before metric E is trusted.

A second surfaced limitation: PR→file linkage depends on the commits recorded against a PR existing in git history at all, which fails for squash/rebase merge workflows (quantified in Module 6, Finding 3).

Module 5 — Bus-factor / maintenance-risk analysis (`analysis/bus_factor.py`, `risk_cli.py`)

Explicit framing: a derived risk *indicator* built from Module 4's activity data, not a fact about any person. Output never states someone "is the only expert"; it reports that available evidence shows certain risk signals, and no threshold has been validated.

Definitions: an **active developer** committed within a configurable window (default 365 days) of the repo's latest commit. **Bus factor** is the smallest number of developers whose combined activity share reaches a coverage threshold (default 50%), computed both over recent activity (`bus_factor_active`) and over all history (`bus_factor_lifetime`). **Departed share** is the fraction of all-time commits made by developers with zero commits in the active window.

Three independent risk **signals**, each recorded with its supporting evidence:

Signal	Triggers when
`single_active_maintainer`	active bus factor is 1 **and** the top active developer holds ≥60% of recent commits
`historical_knowledge_departed`	departed share ≥50%
`low_reviewer_diversity`	≤1 distinct reviewer, only evaluated when ≥3 linked review events exist

Risk level is a plain count of triggered signals (`high` ≥2, `medium` = 1, `low` = 0), plus `dormant` (no recent commits — not the same as "safe") and `insufficient_evidence` (too few commits total). The level rule is intentionally simple so the reasoning stays visible/auditable.

On flask (first result): 5 high, 6 medium, 0 low, 28 dormant, 6 insufficient. Every flagged component's top active developer holds 80–100% of recent commits. Sensitivity analysis found the active-window length matters somewhat; the dominant-share threshold (50/60/70%) changed nothing on flask because flagged components were already ≥80%.

Surfaced limitations: renames split history across old/new paths and understate departed share for the renamed path; "all-or-nothing on one person" risk flags across nearly every active component could

mean the project really is maintainer-concentrated, or that the rule thresholds are too sensitive — not resolvable without a comparison across repositories (which Module 6 provides) and independent judgement.

Module 6 — Multi-repository evaluation (`evaluation/`)

This is the project's main empirical contribution and should likely anchor the "Results" section of the paper. Full tables are in Section 8 below.

Evaluation set: 8 public, non-fork repositories spanning 198–7,713 commits and 4 languages (Python: flask, requests, fastapi, black; Rust: ripgrep; JavaScript: express, ora; Go: cobra). Explicitly *not* a random sample — chosen for variety based on prior expectations of each project's contributor structure — so agreement with expectations is described as weak evidence, not validation. The build plan's suggested target was 10–20 repositories; 8 is framed as a first validation pass, not the final word.

Module 7 — Structured + text retrieval (`retrieval/`)

Answers a natural-language question with citable repository evidence, without calling an LLM (that's Module 8's job). `query.py` does rule-based intent classification (contributors / reviewers / history / why-or-which-PR / risk / issue-links / lookup) and extracts file paths, `#N` references, commit hashes, and developer names; anything unclassified falls back to text search. `structured.py` answers directly from the relational model with every item carrying its SHA or PR/issue number; `text_search.py` runs BM25 over commit messages, PR/issue text, and review comments; `ranking.py` fuses ranked lists via Reciprocal Rank Fusion; `hybrid.py`'s `Retriever.retrieve(question)` ties it together, returning `answers`, ranked `evidence` (each with a stable `ref` like `commit:ab12cd3` or `pr:#123`), and `notes` flagging data gaps inline.

Two notable engineering decisions directly relevant to methodology:

- **Squash-merge recovery at query time.** Where a PR isn't linked to any commit in history (the Module 6 Finding 3 problem), the PR number is recovered from a `(#N)` suffix in the commit subject — GitHub's default squash-merge format — and the resulting answer is labeled `subject` rather than `linked`, so the distinction is preserved rather than silently merged into "found".
- **Honest notes are a first-class output**, not an afterthought: answers state when PR data is mostly unlinkable, when a filename matches several historical paths (a rename), and when a person may be split across a Git name and a GitHub login.

Text-search evaluation (issue title → the PR that closed it, ground truth = GitHub's own "closes #N" links): R@10 ranges 57–81% across 7 repositories against ~0.3–1.1% chance (full table in Section 8). An analysis of *why* queries miss found that only 4–13% of all misses are cases where query and answer share literally zero words (the only failure class embeddings would clearly fix) — most misses share some vocabulary and rank below 10, a ranking problem rather than a vocabulary gap. On this evidence, **embedding-based retrieval was deliberately not implemented**, with the explicit caveat that this doesn't show embeddings wouldn't also help — that comparison wasn't run.

Module 8 — Grounded LLM answers (`llm/`, Gemini)

Pipeline: question → Module 7 retrieval → evidence rendered as text with stable citation ids (`[fact:1]`, `[pr:#123]`, `[commit:ab12cd3]`, `[issue:#7]`) → Gemini, instructed to use only that evidence and cite ids → answer, followed by **mechanical post-hoc checks**: every cited id must be one that was actually shown (else flagged invalid), and any `#number` or commit hash appearing in the answer text that never appeared in the evidence is flagged unverified. A key design point: **if retrieval finds no evidence, the model is never called** — a fixed "not enough evidence" response is returned instead, closing a concrete failure mode found during testing (generic words like "pull request" were matching 8 unrelated PRs and reaching the model, which happened to abstain correctly but shouldn't have been asked at all).

Status is explicitly partial: built and unit-tested with a fake model/HTTP layer; run live on 3 hand-picked questions, and on a 10-question generated benchmark (2 each: top committer of a file, top reviewer of a file, what a PR is about, which PR closed an issue, plus unanswerable controls) — 10/10 correct, 0 invalid citations, 0 unverified numbers, but the authors are explicit that this is "small and easy": templated lookup questions on one repository with one model, saying little about open-ended "why" questions. The formal `--eval` re-run after a bug fix was cut off by Google's free-tier quota before completion.

Untrusted text (from PRs/issues/commits) is placed inside an explicit `<evidence>` block with an instruction to ignore any directives found there — stated as a mitigation, not a guarantee, which is worth citing directly if the paper discusses prompt-injection risk in RAG-over-user-data systems.

Module 9 — Context-aware code explanation (`explain/`)

Explains a specific function or line range together with *only the history that touched those exact lines* — found via `git log -L start,end:path`, which follows a precise line range through history — rather than a file-wide history dump. For flask's `flash` function: 11 commits touch its lines vs. 93 for the whole of `helpers.py`, i.e. the evidence set shrinks by roughly 8× while staying specific to the question asked.

A no-LLM **history-coverage measurement** (`--stats`) sampled Python functions across 4 repositories and found: a function's history is 4–24% of its file's history (so line-level targeting is doing real work); every commit git finds is also present in the database (clone and DB agree); most commits reach a PR number (86–92%, boosted by the `(#N)` subject recovery, most valuable exactly in the squash-merge repos where stored PR links are nearly empty); reaching a *closed issue* varies by project habit (12% requests vs. 77% black). As of the last doc update, **no live model explanation had been generated or read** — blocked on Gemini free-tier quota — so the module's prompt/evidence construction is verified, but its output quality is explicitly marked unvalidated.

Module 10 — Engineering assistance (`assist/`)

A pre-change briefing — not a code suggester — combining: Module 9's line-level change history; commits/PRs that **look like fixes** (keyword rules over subject/title text, or a closed bug-labeled issue, with the matched reason always shown); **co-change** partners (how often another file changed in the same commit, historically, with test files and housekeeping files like changelogs/lockfiles/CI config broken out separately); open issues textually mentioning the file; and the people and Module 5 risk

signals involved. The model is explicitly told to treat co-change as *association, not dependency*, and to say "looks like a fix" rather than "is a fix."

Two no-LLM evaluations, both time-split (70% train / 30% test by commit time):

- **Co-change prediction**: predict a changed file's top-5 historical co-change partners, check against what actually changed together. Beats a "5 most-frequently-changed files" baseline in 6 of 7 repositories tested (by 20–36 points of hit@5), but **loses in express** (47% vs. 49%) — reported as a genuine negative result, not smoothed over.
- **Fix-wording vs. GitHub bug labels**: precision ranges widely (47–94% across repos) and recall is generally low (35–86%), explicitly characterized as "a noisy hint," with the caveat that the wording rule was tuned by looking at these same labels, so there is no held-out validation of it.

As with Module 9, the Gemini-dependent briefing step itself had not been run live at time of writing (quota-blocked); only the evidence/prompt construction is verified.

Module 11 — Web interface (`ui/`)

A local (127.0.0.1-only, no login) Flask app with six screens — Overview, Risk, Knowledge map, Ask, Code, History — over everything built in Modules 0–10. Deliberate design choices worth citing for a "system design" section: the AI step is opt-in on every action (off by default, with full evidence still shown either way); AI answers display their own citation checks as warnings rather than hiding them; all user/model-generated text is inserted via text nodes only (no `innerHTML`, closing an XSS vector); repository names submitted through the UI are server-side validated and never passed to a shell. 105 automated tests plus a documented manual pass across all six screens are recorded in the doc, including specific bugs found and fixed during that pass (unclear messaging on backwards line ranges, missing truncation notices, a technical validation message surfaced to end users).

5. Data model summary

Entity	Key	Notes
Repository	`repository_id` (`owner/name`)	
Developer	`developer_id`	Identity-resolution output; carries confidence + sources
File	`file_id`	Deduped by `(repository_id, path)`
Commit	`commit_id` (= SHA)	`raw_source_reference` = the SHA itself
CommitFile	`commit_file_id`	Commit↔File edge, with additions/deletions
Issue	`issue_id`	`raw_source_reference = issues.jsonl#<number>`
PullRequest	`pr_id`	Carries `commit_shas` and `closing_issue_numbers`
Review	`review_id`	Belongs to a `pr_id`
Label	`label_id`	

Provenance chain, enforced throughout: **raw JSONL record (keyed by SHA or GitHub number) → normalized record (`raw_source_reference`) → relational row → retrieval `ref` → LLM citation id.** Every number the paper reports can in principle be traced back to this chain.

6. Technology choices and why (useful for a "methodology" section)

- **SQLite**, not a graph database, chosen deliberately — the build plan's explicit reasoning was "do not introduce a graph database merely because the project is graph-like; first prove the relationship model with a relational representation."
- **Gemini** (`gemini-3.8-flash`) behind a small provider interface (`llm/client.py`), chosen pragmatically (a prior Google Cloud project's key was rejected for a different model tier; a different project's key worked) — the interface exists so another provider could be substituted.
- **BM25 lexical search**, not embeddings, for Module 7 — a measured, not assumed, decision (Section 4, Module 7 above).
- **Rule-based query interpretation**, not an LLM, for turning a question into a retrieval plan in Module 7 — keeps the expensive/stochastic LLM call confined to final answer generation.

7. What NOT to claim (explicit project guardrails, worth stating in the paper's limitations section)

Carried from the original build plan and enforced throughout the codebase (`docs/PROJECT.md`, `CLAUDE.md`):

- Commit count is never equated with expertise.
- No concentration/risk score is presented as a fact about a person — all are framed as evidence-derived signals or hypotheses.
- Unresolved developer identities are preserved, never silently deleted or guessed into a merge.
- Bots are flagged, not removed, from the record.
- Raw data is immutable; every derived artifact (normalized entities, the SQLite DB, metrics JSON) is regeneratable from it.

8. Full experimental results (Module 6 + Module 7/8 evaluations)

8.1 Dataset and pipeline validity (8 repositories)

repo	commits	span	files	committer identities	bot identities	bot commit share	issues	PRs	reviews	validation
flask	5,557	2010–2026	642	873	3	3%	2,767	2,910	840	OK
requests	6,495	2011–2026	466	803	1	1%	4,183	3,095	1,953	OK
fastapi	7,713	2018–2026	5,449	918	6	44%	3,553	6,359	8,643	OK
ripgrep	2,287	2016–2026	467	497	1	0%	1,828	1,167	920	OK
black	2,346	2018–2026	939	582	3	9%	2,822	2,469	3,948	OK
express	6,170	2009–2026	905	394	1	1%	4,072	2,666	2,204	OK
cobra	1,106	2013–2026	134	345	1	3%	1,259	1,217	1,731	OK
ora	198	2016–2026	24	57	0	0%	155	100	91	OK

All 8 ran end-to-end with zero structural validation failures. Caveat: validation is structural only (no dangling foreign keys); relationship *correctness* was spot-checked by hand on flask only, not sampled systematically.

8.2 Identity resolution and Git↔GitHub linkage

repo	human commits linked to a GitHub login	top-10 committers linked	PRs linked to file changes	reviews linked to file changes	merge commits	commits with no file list
flask	2%	0/10	48%	51%	31%	0%
requests	3%	1/10	45%	44%	25%	1%
fastapi	14%	4/10	0%	0%	0%	0%
ripgrep	4%	2/10	5%	3%	3%	0%
black	25%	5/10	0%	0%	0%	0%
express	1%	0/10	10%	15%	8%	0%
cobra	10%	1/10	10%	0%	10%	0%
ora	4%	1/10	0%	0%	0%	0%

This table is the strongest single piece of empirical evidence for RQ3: linkage is weak (1–25%) in *every* repository tested, meaning for most people, commit activity and PR/review/issue activity sit under

different identity records. The likely cause of the 0% PR-linkage repos (fastapi, black, ora) is squash merging — inferred from 46–79% of commit subjects ending in GitHub's `(#PR-number)` squash format — stated as inferred, not confirmed.

8.3 Concentration metric behaviour

repo	ranked files	median effective contributors (files)	files ≥80% top-share	ρ commits-vs-recency	ρ commits-vs-multi-signal	ranked components	components ≥80% top-share
flask	452	2.4	28%	0.66	0.91	39	36%
requests	304	3.1	9%	0.53	0.89	25	8%
fastapi	1,697	2.0	29%	0.80	0.87	128	33%
ripgrep	191	1.4	58%	0.93	0.88	50	62%
black	214	5.0	6%	0.76	0.90	25	0%
express	492	1.8	44%	0.73	0.94	75	43%
cobra	88	5.0	2%	0.64	0.88	9	0%
ora	9	3.3	22%	0.50	0.85	3	67%

Concentration varies in the expected direction across projects (ripgrep and express most concentrated; black and cobra most distributed). The high commits-vs-multi-signal correlation (0.85–0.94) is explicitly flagged as **not** evidence that the multi-signal metric is better — with PR/review signals largely missing (Section 8.2), metric E is built almost entirely from the same commit data as metric A, so high agreement is expected "by construction."

8.4 Sensitivity analysis (Spearman ρ of file-level concentration rankings)

repo	half-life 90d vs 365d	365d vs 1095d	with vs without bots	files whose top committer changes w/ bots	conservative vs name-merged identities	identities merged away
flask	0.81	0.87	1.00	0/452	1.00	136
requests	0.61	0.69	0.99	3/304	0.99	101
fastapi	0.79	0.92	0.96	14/1697	1.00	26
ripgrep	0.92	0.97	1.00	0/191	1.00	17
black	0.75	0.85	0.97	16/214	0.99	16
express	0.85	0.88	1.00	2/492	1.00	39
cobra	0.56	0.81	1.00	4/88	0.98	19
ora	0.28	0.82	1.00	0/9	1.00	0

The recency half-life is the most influential setting tested; bot handling and *same-name* identity merging barely move rankings. Important methodological caveat carried from the source doc: the identity-merging test here only merges identities sharing a display name, so it does **not** test the cross-

source (Git↔GitHub) linkage problem shown to be the real gap in Section 8.2 — "ranking is stable to what was tested, not to identity resolution in general."

8.5 Risk signal stability

repo	high	medium	low	dormant	insufficient	level agreement @180d	level agreement @730d
flask	5	6	0	28	6	91%	100%
requests	7	5	0	13	5	100%	58%
fastapi	8	88	18	14	26	13%	82%
ripgrep	1	12	2	35	13	40%	80%
black	2	15	3	5	4	50%	85%
express	6	4	0	65	9	50%	70%
cobra	1	3	0	5	1	50%	50%
ora	0	2	0	1	0	100%	100%

Risk-level classification is highly sensitive to the "active" window choice in several repositories (fastapi agreement drops to 13% between a 365-day and 180-day window), meaning **any reported count of high/medium risk components must always be quoted together with the active-window parameter** — a direct, reportable methodological finding.

8.6 Text-retrieval evaluation (issue title → closing PR, R@k, MRR)

repo	queries	corpus (PRs)	R@1	R@5	R@10	MRR	R@10 (title+body)	chance R@10
flask	300	2,910	46%	68%	76%	0.56	71%	0.4%
requests	242	3,095	33%	52%	59%	0.42	54%	0.4%
fastapi	142	6,359	35%	61%	69%	0.47	70%	0.3%
ripgrep	179	1,167	42%	64%	72%	0.52	64%	1.1%
black	400	2,469	34%	51%	57%	0.42	62%	0.5%
express	90	2,666	59%	79%	81%	0.68	77%	0.8%
cobra	195	1,217	44%	66%	73%	0.54	67%	1.0%

8.7 LLM grounding evaluation (flask, gemini-3.8-flash, 10 generated questions)

Question type	Correct
Top committer of a file	2/2
Top reviewer of a file	2/2
What a PR is about	2/2
Which PR closed an issue	2/2
Unanswerable (must abstain)	2/2

0 invalid citations, 0 unverified numbers/hashes, across 9 model calls (one unanswerable question never reached the model because retrieval found nothing). Framed by the authors as showing the *mechanism* works on simple factual lookups, not as a general answer-quality result.

8.8 Engineering-assistance evaluations (no LLM)

Co-change prediction (hit@5 vs. a popularity baseline):

repo	hit@5 co-change	hit@5 popular-files baseline	co-change precision	baseline precision
flask	50%	30%	17%	8%
requests	73%	53%	31%	23%
fastapi	69%	37%	29%	8%
ripgrep	55%	19%	29%	4%
black	69%	35%	25%	10%
express	47%	49%	22%	14%
cobra	72%	46%	32%	12%

Fix-wording heuristic vs. GitHub bug labels:

repo	bug-labeled PRs caught (recall)	false-fire rate on non-bug PRs	precision
requests	70% of 71	9% of 32	94%
fastapi	69% of 68	29% of 146	52%
ripgrep	48% of 67	15% of 71	74%
black	35% of 414	6% of 266	91%
express	86% of 50	53% of 92	47%
cobra	42% of 36	8% of 63	75%

9. Known limitations (consolidated — cite directly in the paper's Limitations section)

1. **Identity linkage across sources is the single largest unresolved gap.** 1–25% of human commits link to a GitHub login across all 8 repositories; its quantitative effect on concentration/risk metrics is explicitly unmeasured.
2. **PR→file linkage fails under squash/rebase merge workflows** — 0% in 3 of 8 repos tested, meaning thousands of PRs and reviews contribute nothing to file-level signals in those repositories. A `(#N)` subject-line recovery heuristic mitigates this at *query* time (Module 7) but the underlying stored linkage is unchanged.
3. **No ground truth exists for concentration or risk scores.** All current results describe internal consistency and sensitivity, not validated accuracy against an independent measure of "real" risk or expertise.
4. **RQ5 (does concentration predict future maintenance problems) has not been tested** — doing so needs a temporal backtest, listed as explicit future work.

5. **Default branch only; renames not tracked** — splits a file's history across old/new paths (concretely confirmed for flask's `flask/` → `src/flask/` move) and is suspected, but not checked, to affect other repositories.
6. **The evaluation set (8 repos) is not a random or representative sample** — chosen for variety based on prior expectations, with the build plan's own suggested target being 10–20.
7. **Gemini-dependent modules (8, 9, 10, and the UI's AI step) are validated only on small samples** due to free-tier quota limits; code explanation and engineering-assistance LLM output specifically had not been read by a human at time of writing.
8. **Fix-detection wording rule was tuned by inspecting the same labels used to evaluate it** — no held-out validation.

10. Suggested paper structure (mapping sections above to a typical IMRaD/engineering-thesis outline)

- **Abstract / Introduction** → Sections 1–2
- **Related work** → not covered in these docs; needs separate literature review (commit-mining tools, bus-factor literature, RAG-over-code systems)
- **System design / Architecture** → Sections 3, 6
- **Methodology** → Section 4 (module-by-module), Section 5 (data model)
- **Experimental setup** → Section 8.1 (the 8-repo set and why)
- **Results** → Sections 8.2–8.8
- **Discussion** → Section 2 (RQ-by-RQ), Section 9
- **Limitations and threats to validity** → Section 9
- **Future work** → the "suggested next steps" already identified per-module: recovering squash-merge PR links at the data layer (not just query time), improving Git↔GitHub identity linking and measuring its effect, a temporal backtest for RQ5, extending to 10–20+ repositories, and completing the Gemini-dependent evaluations once quota allows.
- **Conclusion** → restate the RQ status table (Section 2) as the paper's headline contribution: a working, evaluated pipeline plus a precise, quantified account of where it currently falls short.